SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	A. Govardhan Reddy	**Reg. No.:**	192421359
Date:	15-08-2026	**Duration:**	60 minutes

Code of Conduct

I A. Govardhan Reddy (192421359) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Govardhan Reddy

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

CI/CD Pipeline Design Document

Intelligent Airport Runway Safety and Aircraft Ground Operations Platform

1. Requirement & Pipeline Analysis

The platform ingests continuous, high-frequency data from aircraft position sensors, ground vehicle trackers, ADS-B feeds and weather systems, fuses it in near real time to detect runway incursions, and pushes predictive risk alerts to controllers, ground crews and safety officers. Because it directly informs safety-critical decisions, the CI/CD pipeline must prioritize reliability, low-latency validation, and zero-downtime releases over release speed alone. Key CI/CD requirements derived from the problem statement:

- **Safety-critical reliability:** every change to the incursion-detection or predictive risk-alert logic must pass rigorous automated testing and a manual safety sign-off before reaching production.
- **Polyglot microservices:** separate services for tracking, incursion detection, predictive analytics, ground-ops dashboard and incident logging need independent build/test/deploy pipelines.
- **Real-time data integration:** the pipeline must validate integration with simulated ADS-B, ground-vehicle and weather data streams before any live deployment.
- **Scalability across airports:** pipeline artifacts (Docker images/Helm charts) must be environment-agnostic so the same build can scale from a small regional airport to a major international hub.
- **Role-based access & data security:** controller, ground-crew and safety-officer dashboards handle operationally sensitive data, requiring strict RBAC and vulnerability scanning in the pipeline.
- **High availability:** runway safety monitoring cannot tolerate downtime, mandating blue-green/canary deployment and automated rollback.
- **Auditability:** since incident/near-miss logging feeds safety audits, all pipeline actions and deployments must be traceable and logged.

2. Pipeline Architecture & Workflow

The pipeline is organized into six sequential stages — Source & Commit, Continuous Integration, Package, Test Environment, Staging, and Production Deployment — with automated notifications and rollback wired across every gate. The diagram below shows the full flow from code commit to production release.

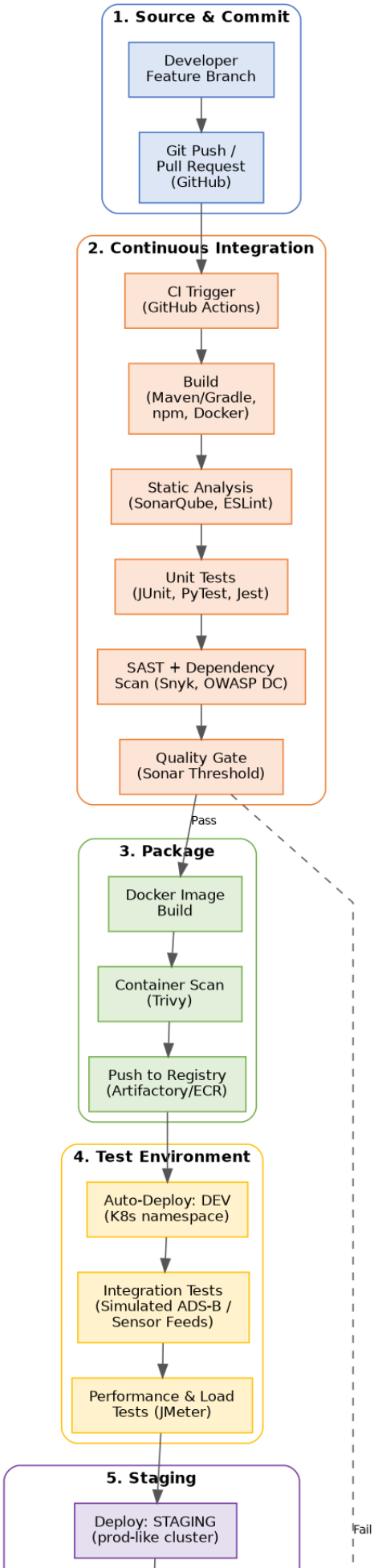
1. Source & Commit
Developer Feature Branch
Git Push / Pull Request (GitHub)
2. Continuous Integration
CI Trigger (GitHub Actions)
Build (Maven/Gradle, npm, Docker)
Static Analysis (SonarQube, ESLint)
Unit Tests (JUnit, PyTest, Jest)
SAST + Dependency Scan (Snyk, OWASP DC)
Quality Gate (Sonar Threshold)
Pass
Fail
3. Package
Docker Image Build
Container Scan (Trivy)
Push to Registry (Artifactory/ECR)
4. Test Environment
Auto-Deploy: DEV (K8s namespace)
Integration Tests (Simulated ADS-B / Sensor Feeds)
Performance & Load Tests (JMeter)
5. Staging
Deploy: STAGING (prod-like cluster)

Figure 1: CI/CD Pipeline — Commit to Production Deployment

Workflow Summary

Stage	Key Actions	Trigger
1. Source & Commit	Developer commits to feature branch; opens PR against main	Git push
2. Continuous Integration	Build → static analysis → unit tests → SAST/dependency scan → quality gate	PR opened / merge to main
3. Package	Docker image build → container vulnerability scan → push to registry	CI stage passes quality gate
4. Test Environment (Dev)	Auto-deploy to Dev namespace → integration tests against simulated sensor/ADS-B/weather feeds → performance/load tests	Successful image push
5. Staging	Deploy to production-like cluster → UAT & safety validation by ATC/safety officers → manual approval gate	Dev tests pass
6. Production Deployment	Canary / blue-green rollout → automated health & smoke checks → full rollout → continuous monitoring	Staging approval granted

3. Tools & Technology Selection

Tools are selected for native support of polyglot microservices, container-native scalability, and strong security/observability tooling appropriate for a safety-critical aviation platform.

Stage	Tool / Technology	Justification
Source Code Management	GitHub (Git), trunk-based branching with short-lived feature branches	Enables parallel work across tracking, alerting, dashboard and incident-logging modules while keeping main branch always deployable — critical for a safety system.
CI Orchestration	GitHub Actions (self-hosted runners for airport-network-isolated jobs)	Native integration with GitHub, YAML-based pipelines-as-code, easy to scale runners for multiple microservices.
Build	Maven/Gradle (Java backend services), npm/Webpack (React dashboards), Docker	Airport platforms are typically polyglot microservices; each service builds independently and is containerized for consistent runtime across environments.
Static Code Analysis	SonarQube, ESLint/Checkstyle	Enforces coding standards and detects code smells/bugs early, essential for a safety-critical codebase.
Unit Testing	JUnit 5 / Mockito (Java), PyTest (ML risk-engine services), Jest (dashboard UI)	Framework-native unit testing with mocking of sensor/ADS-B data sources.
Security Scanning	Snyk / OWASP Dependency-Check (SAST & SCA), Trivy (container image scan)	Aviation systems handle sensitive operational data; vulnerabilities in dependencies or images must be caught before deployment.
Artifact Repository	JFrog Artifactory / Amazon ECR	Central, versioned storage of build artifacts and Docker images with promotion between environments.
Container Orchestration	Kubernetes (K8s), Helm charts	Provides scalability across airports of varying size/traffic, self-healing, and rolling/blue-green/canary deployment support.
Integration & API Testing	Postman/Newman, WireMock (simulated ADS-B & weather feeds)	Validates real-time data fusion between aircraft, ground vehicle and weather sources without needing live airside hardware in CI.

Performance/Load Testing	Apache JMeter / k6	Confirms the platform can process high-frequency position updates during peak traffic without degrading incursion-alert latency.
Monitoring & Observability	Prometheus + Grafana, ELK Stack (Elasticsearch, Logstash, Kibana)	Real-time dashboards and log analysis are required to detect anomalies and support the centralized incident-logging objective.
Notifications	Slack / MS Teams webhook, PagerDuty	Immediate alerts to DevOps and safety teams on pipeline failures or production health-check anomalies.
Secrets Management	HashiCorp Vault / AWS Secrets Manager	Protects credentials for radar/ADS-B feeds and weather APIs; supports RBAC required for controller/supervisor/officer roles.

4. Automated Testing Strategy

- **Unit Testing:** Every microservice (tracking, incursion detection, predictive risk engine, dashboard, incident logging) carries unit tests (JUnit/PyTest/Jest) run on every commit, targeting ≥80% code coverage, with mocked sensor/weather inputs.
- **Integration Testing:** Automated tests validate data fusion across aircraft position, ground vehicle, and weather modules using simulated ADS-B/sensor feeds (WireMock) so pipeline runs do not depend on live airside hardware.
- **Regression Testing:** A regression suite re-runs core incursion-detection and alerting scenarios on every merge to main to catch safety-relevant behaviour changes.
- **Performance & Load Testing:** JMeter/k6 scripts simulate peak-traffic conditions (high aircraft/vehicle density) in the Dev/Staging environments to confirm alert latency stays within safety thresholds.
- **Security Testing:** SAST (SonarQube/Snyk) and dependency scanning run in CI; DAST and container scanning (Trivy) run before promotion to Staging.
- **User Acceptance Testing (UAT):** Air traffic controllers, ground-crew supervisors and safety officers validate role-based dashboards and alert workflows in Staging before production approval.
- **Chaos/Failure Testing:** Periodic fault-injection tests (e.g., simulated sensor dropout, network partition) verify the system degrades safely and alerts operators, given the platform's safety-critical nature.

5. Deployment Strategy

Environments

- **Development:** Auto-deployed on every successful build via Kubernetes namespace isolation; used for integration and early performance testing; disposable and rebuilt frequently.
- **Testing/Staging:** A production-like cluster seeded with simulated live-traffic data; hosts UAT and safety validation by controllers and safety officers; gated by manual approval before promotion.
- **Production:** Multi-airport-capable Kubernetes cluster with autoscaling; receives only artifacts that passed Staging approval and automated health checks.

Release Strategy

- **Canary / Blue-Green Deployment:** New versions are first routed to a small percentage of traffic (or a parallel 'green' environment) while automated health and smoke checks run; full cutover happens only after checks pass, ensuring zero-downtime releases for a safety system that cannot go dark.
- **Feature Flags:** Predictive risk-engine model updates are released behind feature flags so new ML models can be validated on live (shadow) traffic before fully replacing the active model.
- **Scalability:** Helm chart values are parameterized per airport size/traffic complexity, allowing the same pipeline artifact to be deployed at regional or hub-scale airports without code changes.

6. Security, Quality Checks, Notifications & Rollback

Security

- RBAC enforced across controller, ground-crew supervisor and safety-officer dashboard roles, provisioned via Vault-managed secrets.
- TLS encryption in transit for all sensor, ADS-B and weather data feeds; secrets never stored in source control.
- SAST, SCA (dependency scanning) and container image scanning gate every build before it can be packaged.

Quality Checks

- SonarQube quality gate blocks merge/build on code-smell, bug, or coverage threshold failures.
- Manual UAT/safety sign-off gate in Staging, performed by safety officers, before production promotion.

Notifications

- Slack/MS Teams webhooks notify the DevOps channel on any pipeline stage failure.
- PagerDuty escalates production health-check failures or monitoring anomalies to on-call engineers immediately.

Rollback Mechanism

- Automated health and smoke checks run immediately after canary/blue-green cutover; a failure automatically routes traffic back to the last stable version.
- Prometheus/Grafana anomaly detection during full production monitoring can also trigger an automated rollback and incident notification, minimizing exposure time for a safety-critical failure.
- All rollbacks are logged to the centralized incident-logging system, feeding the same safety trend-analysis objective the platform is built to support.

7. Conclusion

This pipeline automates the journey from code commit to production for a safety-critical, real-time airport operations platform. Rigorous automated testing (unit, integration, performance, security, chaos), mandatory manual safety validation before production, and blue-green/canary deployment with automatic rollback collectively ensure that runway safety and ground-operations software can be updated frequently and confidently — without compromising the zero-tolerance safety standards of airside operations.